

CubeSat Payload Fragmentation Pipeline

A Complete Walkthrough of Task 1 (FreeRTOS)

Prepared for Hardik Singhal

1 The Big Picture (Why Are We Even Doing This?)

Hardik, your mentor Lucas needs a system that takes data from a satellite instrument (the “payload,” related to an “occultation” event) and gets it down to Earth via radio. The problem: that data can be **big**, but radio packets can only carry a **limited number of bytes** at a time (NGHam’s max is 220 bytes per packet).

So the solution is a **pipeline** — three separate jobs, each one handing its output to the next:

```
[Payload Memory] -> TASK 1 -> TASK 2 -> TASK 3 -> [Radio Transmission]
                (fragment) (queue) (NGHam wrap)
```

Since the real firmware uses **FreeRTOS**, each of those three jobs must be written as an independent **FreeRTOS task** — a piece of code that runs continuously, on its own, at the same time as the others.

Right now, we’ve only built Task 1. Task 2 and Task 3 don’t exist yet — we used a **temporary stand-in** (vTempConsumer) just to prove Task 1 works correctly, exactly like Amrit did.

2 Your File Structure — What Each Thing Actually Is

Let’s open your project folder mentally:

```
cubesat-pipeline/
|-- FreeRTOS-Kernel/ <- Downloaded code (not written by you)
|-- src/ <- YOUR code
| |-- FreeRTOSConfig.h
| |-- main.c
| `-- payload_chunk.h
|-- build/ <- Auto-generated compiled program
`-- Makefile <- Build instructions
```

2.1 FreeRTOS-Kernel/ — The Engine You Didn’t Write

This is the actual FreeRTOS **operating system code**, downloaded from GitHub. You didn’t write any of it — it’s the “engine” that makes tasks and queues actually work. Key files inside it:

- **tasks.c** — the scheduler. This is the code that decides “okay, pause Task 1 now, let Task 2 run for a bit, now switch back.” Every time you called `xTaskCreate()` or `vTaskDelay()`, this file’s code is what actually executes behind the scenes.
- **queue.c** — implements everything `xQueueCreate()`, `xQueueSend()`, `xQueueReceive()` actually do internally (safely storing/copying data, blocking a task until data arrives, etc.)

- `list.c` — a helper data structure (linked lists) that `tasks.c` and `queue.c` use internally to keep track of things like “which tasks are waiting” — you never call this directly.
- `portable/ThirdParty/GCC/Posix/port.c` — **this is the critical adapter**. FreeRTOS was designed for real microcontrollers with real hardware timers. Since we’re running on a Mac (no real MCU), this file “tricks” FreeRTOS into thinking it’s on real hardware by using Mac/Linux’s own thread system (pthreads) to simulate task-switching. This is the file where we had to comment out that one `pthread_setname_np` line, and is also central to why we needed `-lpthread` when linking.
- `portable/MemMang/heap_3.c` — handles memory allocation for tasks/queues. FreeRTOS needs to reserve memory (RAM) for each task’s stack and for queue storage — `heap_3.c` is the specific strategy we chose, which just wraps the normal C `malloc()`/`free()` functions (the simplest option, good for simulation on a computer with plenty of RAM).

2.2 `src/FreeRTOSConfig.h` — The Settings Panel

Every single project that uses FreeRTOS **must** provide this file. It’s not optional — the kernel literally won’t compile without it. It answers questions like:

- How many priority levels do I want to support? (`configMAX_PRIORITIES = 5`)
- How much total memory (heap) can all my tasks/queues use combined? (`configTOTAL_HEAP_SIZE = 128 KB`)
- How often should the scheduler “tick” (check if it’s time to switch tasks)? (`configTICK_RATE_HZ = 1000`, meaning 1000 times per second = every 1 millisecond)
- Should I check for dangerous bugs like stack overflows or failed memory allocations? (We turned these ON — `configCHECK_FOR_STACK_OVERFLOW = 2`, `configUSE_MALLOC_FAILED_HOOK = 1`)

That last point is *exactly* why we had to add those two “hook” functions (`vApplicationMallocFailedHook`, `vApplicationStackOverflowHook`) to `main.c` — we told FreeRTOS “yes, check for these problems,” so FreeRTOS then requires us to supply the function it should call *if* those problems ever happen.

2.3 `src/payload_chunk.h` — Your Custom Data Envelope

This is a file **you (we) wrote** — not part of FreeRTOS at all. It defines the exact shape of one “chunk” of fragmented data:

```
typedef struct {
    uint16_t occultation_id;
    uint8_t sequence_number;
    uint8_t total_chunks;
    uint8_t data_size;
    uint8_t data[32];
} PayloadChunk_t;
```

Think of `PayloadChunk_t` as a **paper form** with 5 blank fields on it. Every time Task 1 creates a chunk, it fills out one of these forms completely, and that whole filled-out form travels through the queue as a single unit. `CHUNK_DATA_SIZE` (defined as 32) controls how big the `data[]` array is — this is the maximum number of actual payload bytes one chunk can hold (matching Amrit’s placeholder choice).

2.4 src/main.c — Your Actual Program Logic

This is where the real “story” of your program lives. Let’s dissect it completely in the next section.

2.5 Makefile — The Build Recipe

This file tells `gcc` (the compiler): “here are all the `.c` files to compile (from `FreeRTOS-Kernel` AND from `src/`), here’s where to find all the necessary `.h` header files (the `-I` flags), and here’s what to link together at the end into one final program called `build/cubesat_pipeline`.” You never run `gcc` by hand — you just type `make`, and it reads this recipe and does dozens of `gcc` calls for you automatically (you can see all of those individual `gcc ... -c ...` lines scroll by every time you ran `make`).

3 Walking Through main.c — Line by Line

Let’s go through exactly what we wrote, and connect every part to what you actually saw in your terminal output.

3.1 The Includes (top of file)

```
#include <stdio.h> // gives us printf()
#include <string.h> // gives us memcpy(), memset()
#include "FreeRTOS.h" // core FreeRTOS types/macros
#include "task.h" // xTaskCreate, vTaskDelay, etc.
#include "queue.h" // xQueueCreate, xQueueSend, xQueueReceive
#include "payload_chunk.h" // our own PayloadChunk_t struct
```

Nothing mysterious here — we’re just pulling in the tools we need from each library.

3.2 The Queue Handle (global variable)

```
QueueHandle_t xChunkQueue;
```

This is like declaring “there will be a mailbox called `xChunkQueue`, but we haven’t built it yet.” It’s global (declared outside any function) so that **both** Task 1 and `TempConsumer` can access the same queue — remember, this is literally the communication channel between them.

3.3 The Fake Event Data (simulating payload memory)

```
typedef struct {
    uint16_t occultation_id;
    uint16_t data_size;
} FakeEvent_t;

static const FakeEvent_t fake_events[] = {
    { .occultation_id = 1000, .data_size = 100 },
    { .occultation_id = 1001, .data_size = 45 },
    { .occultation_id = 1002, .data_size = 70 },
};
```

Since we don't have a real camera/sensor generating real occultation data yet, **we invented 3 fake events** to test with. This directly matches your output:

```
occultation 1000: 100 bytes
occultation 1001: 45 bytes
occultation 1002: 70 bytes
```

This is the “simulated memory” Lucas asked for — instead of reading real hardware, we just hardcoded 3 fake scenarios with different sizes (one exact multiple of 32, one smaller, one in-between) to test that our fragmentation math handles all cases correctly.

3.4 TASK 1 — The Payload Reader/Fragmenter (the heart of it)

Let's go piece by piece.

3.4.1 The outer loop — looping through each fake event

```
for (int e = 0; e < (int)NUM_FAKE_EVENTS; e++)
{
    uint16_t occ_id = fake_events[e].occultation_id;
    uint16_t data_len = fake_events[e].data_size;
```

This just walks through our 3 fake events one at a time: first `occ_id=1000`, `data_len=100`, then `occ_id=1001`, `data_len=45`, then `occ_id=1002`, `data_len=70`.

3.4.2 Creating fake payload bytes

```
uint8_t fake_payload[256];
for (int i = 0; i < data_len; i++)
{
    fake_payload[i] = (uint8_t)(0x10 + (e * 0x30) + i);
}
```

Since we don't have real camera data, we generate **fake bytes with a predictable pattern** — starting at some base value (which shifts a bit for each event, using $e * 0x30$) and counting upward. This is purely so that when we print `first_byte=0x10`, `first_byte=0x40`, etc. in the output, we can visually confirm “yes, this chunk really does contain the data I expect, in the right order” — it's a debugging trick, not anything that matters for real payload data later.

This explains your output:

- Event 0 (occ 1000): bytes start at `0x10` → first chunk shows `first_byte=0x10` ✓
- Event 1 (occ 1001): bytes start at $0x10 + 1 * 0x30 = 0x40$ → shows `first_byte=0x40` ✓
- Event 2 (occ 1002): bytes start at $0x10 + 2 * 0x30 = 0x70$ → shows `first_byte=0x70` ✓

3.4.3 Calculating how many chunks we need

```
uint8_t total_chunks = (data_len + CHUNK_DATA_SIZE - 1) / CHUNK_DATA_SIZE;
```

This is a common C trick called “**ceiling division**” — normal integer division in C always rounds DOWN (e.g., $100 / 32 = 3$, throwing away the remainder), but we need to round UP (since even 1 leftover byte still needs its own chunk). Adding `CHUNK_DATA_SIZE - 1` (i.e., 31) before dividing forces it to round up correctly.

Let’s verify with your real numbers:

- 100 bytes: $(100 + 31)/32 = 131/32 = 4$ (integer division truncates) → **4 chunks** ✓ (matches your output exactly!)
- 45 bytes: $(45 + 31)/32 = 76/32 = 2$ → **2 chunks** ✓
- 70 bytes: $(70 + 31)/32 = 101/32 = 3$ → **3 chunks** ✓

3.4.4 The inner loop — building each individual chunk

```
int bytes_remaining = data_len;
int offset = 0;

for (uint8_t seq = 1; seq <= total_chunks; seq++)
{
    PayloadChunk_t chunk;
    memset(&chunk, 0, sizeof(chunk));

    uint8_t this_chunk_size = (bytes_remaining < CHUNK_DATA_SIZE)
        ? (uint8_t)bytes_remaining
        : CHUNK_DATA_SIZE;
```

Let’s trace this exactly for **occultation 1000 (100 bytes)**:

Iteration (seq)	bytes_remaining (before)	this_chunk_size	bytes_remaining (after)
1	100	$\min(100, 32) = \mathbf{32}$	68
2	68	$\min(68, 32) = \mathbf{32}$	36
3	36	$\min(36, 32) = \mathbf{32}$	4
4	4	$\min(4, 32) = \mathbf{4}$	0

This exactly matches your output: seq=1/4 bytes=32, seq=2/4 bytes=32, seq=3/4 bytes=32, seq=4/4 bytes=4. The last chunk is smaller because only 4 bytes were left over — the code correctly detects “don’t take a full 32, there’s only 4 remaining” using that `bytes_remaining < CHUNK_DATA_SIZE` check.

3.4.5 Filling in the PayloadChunk_t “form”

```
chunk.occultation_id = occ_id;
chunk.sequence_number = seq;
chunk.total_chunks = total_chunks;
chunk.data_size = this_chunk_size;
memcpy(chunk.data, &fake_payload[offset], this_chunk_size);
```

This is literally filling out the 5 blank fields of our `PayloadChunk_t` struct (from Section 2.3):

- `occultation_id` = which event (e.g., 1000)
- `sequence_number` = which piece (e.g., 1, 2, 3, or 4)
- `total_chunks` = how many pieces exist total (e.g., 4)
- `data_size` = how many actual bytes are in THIS chunk (32, or 4 for the last one)
- `data` = a **copy** of the actual bytes, taken from the right spot (`offset`) in our fake payload array

`memcpy` here is doing the actual byte-copying — it reaches into `fake_payload` starting at position `offset`, and copies `this_chunk_size` bytes into the chunk's data array.

3.4.6 Sending the chunk into the queue

```
xQueueSend(xChunkQueue, &chunk, portMAX_DELAY);

offset += this_chunk_size;
bytes_remaining -= this_chunk_size;

vTaskDelay(pdMS_TO_TICKS(50));
```

`xQueueSend` copies the entire filled-out `chunk` struct into the queue's internal storage. Then we move our `offset` pointer forward (so next iteration reads the *next* section of `fake_payload`) and reduce `bytes_remaining` accordingly. The small `vTaskDelay(50)` just simulates “this takes a little bit of real work time” — and crucially, it's also what lets FreeRTOS switch over and let `TempConsumer` actually run and drain the queue, instead of Task 1 hogging everything.

3.4.7 After all events are done

```
printf("[Task1-PayloadReader] all occultations fragmented. Task finished.\n");
```

This is exactly the last line you saw before `TempConsumer` announced it stopped receiving anything.

3.5 The Temporary Consumer — `vTempConsumer`

```
if (xQueueReceive(xChunkQueue, &received, pdMS_TO_TICKS(3000)) == pdTRUE)
{
    total_received++;
    printf("[TempConsumer] got chunk occ=%d seq=%d/%d bytes=%d first_byte=0x%02X\n", ...);
}
else
{
    printf("[TempConsumer] no more chunks arriving. Total chunks received: %d\n", total_received);
    ;
    ...
}
```

This task does one simple thing forever: **wait up to 3 seconds for a new chunk to arrive** in the queue. If one arrives, print its details and count it. If **3 full seconds pass with nothing arriving** (meaning Task 1 must be finished producing), print the final total and go idle.

This is exactly why your output ends with:

```
[TempConsumer] no more chunks arriving. Total chunks received: 9
```

9 because: 4 (event 1000) + 2 (event 1001) + 3 (event 1002) = **9 total chunks**, which is mathematically exactly correct.

3.6 main() — Where Everything Gets Wired Together

```
xChunkQueue = xQueueCreate(10, sizeof(PayloadChunk_t));

xTaskCreate(vTask1_PayloadReader, "Task1", configMINIMAL_STACK_SIZE * 2, NULL, 1, NULL);
xTaskCreate(vTempConsumer, "TempConsumer", configMINIMAL_STACK_SIZE * 2, NULL, 1, NULL);

vTaskStartScheduler();
```

1. **Build the mailbox first** — `xQueueCreate(10, sizeof(PayloadChunk_t))` means “make a queue that can hold up to 10 `PayloadChunk_t` items at once.”
2. **Register both tasks** — tell FreeRTOS “these two functions should run as independent tasks.”
3. **Start the engine** — `vTaskStartScheduler()` hands control over to FreeRTOS forever; from this point on, FreeRTOS is automatically switching between Task 1 and TempConsumer every time one of them calls a delay or a blocking queue operation.

3.7 The Hook Functions (bottom of file)

These 3 functions (`vAssertCalled`, `vApplicationMallocFailedHook`, `vApplicationStackOverflowHook`) aren’t part of our pipeline’s actual logic — they’re **safety nets** required because we turned on certain checks in `FreeRTOSConfig.h`. They never actually ran in your test (no errors occurred), but they must exist or the program wouldn’t even link/compile, as you saw earlier.

4 What Is and Isn’t “Real” Right Now

Part	Real or Simulated?
FreeRTOS tasks/scheduling	Real — this is genuine FreeRTOS logic, will work unchanged on the real MSP430 later
Queue behavior	Real — same code will run unchanged later
Payload data (<code>fake_payload[]</code>)	Simulated — fake, made-up bytes; later this gets replaced with actually reading from real payload memory
The 3 occultation events	Simulated — invented for testing; later these come from a real instrument/event trigger
The microcontroller (Mac instead of MSP430)	Simulated — we’re using the POSIX port; later this needs recompiling for MSP430
NGHam wrapping	Not built yet — this is Task 3, still ahead of us

5 Connecting Back to the Theory (NGHam — for later)

Right now, our `PayloadChunk_t` has 32 bytes of data max — this is a **placeholder**, not tied to NGHam yet. When we build **Task 3** later, each chunk's `data` field (up to 32 bytes) will become the **Payload** field inside an actual NGHam packet (recall from Part A of our documentation: NGHam payload can be up to 220 bytes — so our 32-byte chunks would easily fit in even the smallest NGHam Size Tag category, “Size 1,” which allows up to 28 bytes...actually wait, $32 > 28$, so we'd need at least “Size 2” which allows up to 60 bytes). This is actually one of the open questions Amrit raised to Lucas — **is 32 bytes the right chunk size**, given NGHam's specific size tiers? Something to keep in mind once we reach Task 3.

Does this fully make sense now, bro? Ask about any specific part that's still fuzzy — don't move to Task 2 until you feel solid on this.